

Configuring a Network ACL (NACL)

To Begin with the Lab

Summary of the Lab

Note: This lab assumes that a VPC already exists with two EC2 instances running in the same subnet. Both instances use a Security Group that allows **All Traffic**, so the focus of this lab is on understanding and configuring **Network ACLs (NACLs)**.

Open the AWS Management Console and navigate to the VPC service. Identify the subnet containing the two EC2 instances and verify that both instances are accessible through SSH, Ping, and HTTP before applying any Network ACL rules.

Open the Network ACLs section and observe that the subnet is currently associated with the Default Network ACL. By default, the Default Network ACL allows all inbound and outbound traffic, which is why the EC2 instances are fully accessible.

acl-0e9a80c0ee9c649b9 Actions ▾

Details Info

Network ACL ID
acl-0e9a80c0ee9c649b9

Owner
463646775279

Associated with
2 Subnets

Default
Yes

VPC ID
vpc-099792461c7d0a616

Inbound rules | Outbound rules | Subnet associations | Tags

Inbound rules (2) Edit inbound rules

Rule number	Type	Protocol	Port range	Source	Allow/Deny
100	All traffic	All	All	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

Click **Create Network ACL**, provide a name such as **Test**, select the existing VPC, and create the Network ACL. At this stage, the new Network ACL is not associated with any subnet and therefore does not affect any resources.

Create network ACL Info

A network ACL is an optional layer of security that acts as a firewall for controlling traffic in and out of a subnet.

Network ACL settings

Name - optional
Creates a tag with a key of 'Name' and a value that you specify.

VPC
VPC to use for this network ACL.

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Value - optional

Remove tag

Add tag

You can add 49 more tags

Cancel Create network ACL

Open the **Subnet Associations** tab of the newly created Network ACL and click **Edit Subnet Associations**. Select the subnet containing the EC2 instances and save the changes so that the subnet starts using the new Network ACL.

Edit subnet associations [Info](#)
Change which subnets are associated with this network ACL.

Available subnets (1/2)
Filter subnet associations

Name	Subnet ID	Associated with	Availability Zone	IPv4 CIDR	IPv6 CIDR
-	subnet-0d10d67b324b6c997	acl-0e9a80c0ee9c649b9	use1-az6 (us-east-1b)	172.31.32.0/20	-
☒	subnet-0b5783ef03796c671	acl-0e9a80c0ee9c649b9	use1-az4 (us-east-1a)	172.31.16.0/20	-

Selected subnets
subnet-0b5783ef03796c671 X

Cancel Save changes

After associating the subnet with the new Network ACL, verify that you can no longer access the EC2 instances using **SSH**, **Ping**, or **HTTP**. This happens because a newly created Network ACL denies all inbound and outbound traffic by default.



Open the **Inbound Rules** section of the Network ACL and click **Edit Inbound Rules**. Add a new rule with **Rule Number 100**, allow **ICMP (Ping)** traffic from **0.0.0.0/0**, and save the changes.

VPC > Network ACLs > acl-0d0e641a49a4940be / test > Edit inbound rules

Edit inbound rules [Info](#)
Inbound rules control the incoming traffic that's allowed to reach the VPC.

Rule number	Type	Protocol	Port range	Source	Allow/Deny
100	Custom ICMP - IPv4	All	N/A	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

Add new rule Sort by rule number

Cancel Preview changes Save changes

Open the **Outbound Rules** section and edit the rules by adding **Rule Number 100** that allows **All Traffic** to **0.0.0.0/0**. Since Network ACLs are **stateless**, both inbound and outbound rules must be configured for traffic to work correctly.

VPC > Network ACLs > acl-0d0e641a49a4940be / test > Edit outbound rules

Edit outbound rules [Info](#)
Outbound rules control the outgoing traffic that's allowed to leave the VPC.

Rule number	Type	Protocol	Port range	Destination	Allow/Deny
100	All traffic	All	All	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

Add new rule Sort by rule number

Cancel Preview changes Save changes

Test the connection by pinging the EC2 instance again. The ping request should now succeed because ICMP traffic is allowed in the inbound direction and all outbound responses are permitted.

```

amanr@LAPTOP-7OU8O5T1 MINGW64 ~
$ ping 107.23.121.122

Pinging 107.23.121.122 with 32 bytes of data:
Reply from 107.23.121.122: bytes=32 time=211ms TTL=114
Reply from 107.23.121.122: bytes=32 time=213ms TTL=114
Reply from 107.23.121.122: bytes=32 time=211ms TTL=114
Reply from 107.23.121.122: bytes=32 time=211ms TTL=114

Ping statistics for 107.23.121.122:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 211ms, Maximum = 213ms, Average = 211ms

amanr@LAPTOP-7OU8O5T1 MINGW64 ~
$ ping 98.88.70.184

Pinging 98.88.70.184 with 32 bytes of data:
Reply from 98.88.70.184: bytes=32 time=213ms TTL=114
Reply from 98.88.70.184: bytes=32 time=215ms TTL=114
Reply from 98.88.70.184: bytes=32 time=213ms TTL=114
Reply from 98.88.70.184: bytes=32 time=214ms TTL=114

Ping statistics for 98.88.70.184:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 213ms, Maximum = 215ms, Average = 213ms

amanr@LAPTOP-7OU8O5T1 MINGW64 ~

```

Return to the **Inbound Rules** and add another rule with **Rule Number 200** to allow **SSH (TCP Port 22)** from **0.0.0.0/0**. Save the changes and verify that you can now establish an SSH connection to the EC2 instance.

VPC > Network ACLs > acl-0d0e641a49a4940be / test > Edit inbound rules

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the VPC.

Rule number Info	Type Info	Protocol Info	Port range Info	Source Info	Allow/Deny Info	
100	All ICMP - IPv4	ICMP (1)	All	0.0.0.0/0	Allow	Remove
200	SSH (22)	TCP (6)	22	0.0.0.0/0	Allow	Remove
*	All traffic	All	All	0.0.0.0/0	Deny	

[Add new rule](#) [Sort by rule number](#)

[Cancel](#) [Preview changes](#) [Save changes](#)

Now you can see we are able to connect to the instance

